Department of Computer Engineering

CO2203 – Object Oriented Programming

Group Project Specification

University Course Registration, Timetable & Attendance Management System

Module	CO2203 – Object Oriented Programming (C++)
Assessment type	Group project (3 members per group)
Weight	XX% of the final module mark
Duration	4 weeks
Design checkpoint	End of Week 2 – Project plan and UML class diagram
Final submission	Week 4, Day 3 – Source code, report, and updated UML
Viva / demonstration	End of Week 4 – 20-minute demonstration and individual viva

1. Project Overview

Your group will design and implement a console-based University Course Registration, Timetable & Attendance Management System in C++. The system allows students to register for courses subject to prerequisite and capacity rules, automatically detects timetable clashes, lets lecturers manage course offerings and record class attendance, and lets administrators oversee the whole system. All data must persist between program runs through file storage.

Attendance capture is deliberately open-ended: the specification fixes the attendance core (sessions, records, validation, and reporting) but requires you to implement the capture mechanism behind an abstract AttendanceCapture interface and choose your own concrete mechanism from an approved list (Section 3.7). Your choice, and its justification, form part of the design checkpoint.

The purpose of this project is not merely to produce a working program, but to demonstrate disciplined object-oriented design. You will first design your system as a UML class diagram, defend that design at a checkpoint, and then implement it. Your final code will be checked for conformance against your approved design, and any deviations must be justified in a design change log.

2. Learning Outcomes Assessed

On successful completion of this project, you will be able to:

1. Design an object-oriented solution for a realistic problem using UML class diagrams.
2. Implement class hierarchies in C++ using inheritance, polymorphism, and abstract classes.
3. Apply encapsulation, composition, and aggregation appropriately in class design.
4. Manage dynamic resources correctly using constructors, destructors, and the Rule of Three/Five.
5. Use operator overloading, exception handling, templates, and the STL effectively.
6. Persist and restore object state using file handling.
7. Work collaboratively in a team with clearly defined interfaces and responsibilities.

3. System Description and Functional Requirements

The system must support three user roles: Student, Lecturer, and Administrator. Each role logs in and sees a role-specific menu. The minimum functional requirements are listed below; groups are encouraged to add well-designed extensions.

3.1 User Management

- FR1.1 – The system shall support login for three user roles: Student, Lecturer, and Administrator.
- FR1.2 – The Administrator shall be able to create, update, and remove user accounts.
- FR1.3 – Each role shall see a different dashboard/menu determined polymorphically (not by chains of if/else on a role string).

3.2 Course Management

- FR2.1 – The Administrator shall be able to create, edit, and remove course offerings, each with a code, title, credit value, capacity, assigned lecturer, and prerequisites.
- FR2.2 – The system shall support at least three course types (e.g., lecture-based, lab-based, and project-based courses) with differing credit or grading behaviour implemented through a class hierarchy.
- FR2.3 – A Lecturer shall be able to view the enrolment list of their own courses only.

3.3 Enrolment Engine

- FR3.1 – A Student shall be able to enrol in and drop courses.
- FR3.2 – Enrolment shall be rejected with a meaningful message when prerequisites are not met or the course is full.
- FR3.3 – These rule violations shall be signalled using custom exception classes.
- FR3.4 – Prerequisite chains (a course whose prerequisite has its own prerequisite) shall be checked correctly.

3.4 Timetable and Clash Detection

- FR4.1 – Each course shall have one or more weekly time slots (day, start time, end time, location).
- FR4.2 – The system shall reject an enrolment that creates a timetable clash with the student's existing courses.
- FR4.3 – A Student shall be able to view their personal weekly timetable.
- FR4.4 – Clash detection between two time slots shall be implemented via an overloaded operator (e.g., operator== or a dedicated overlaps operator).

3.5 Persistence

- FR5.1 – All users, courses, and enrolments shall be saved to files and reloaded on startup, preserving all relationships between objects.
- FR5.2 – The file format is your design decision, but loading and saving must be implemented through a clean, encapsulated interface (e.g., a Repository or Storage class), not scattered file code.
- FR5.3 – Corrupt or missing data files shall be handled gracefully using exceptions.

3.6 Reporting

- FR6.1 – The Administrator shall be able to generate at least one report (e.g., enrolment summary per course).
- FR6.2 – Printing of domain objects (students, courses, timetables) shall use an overloaded stream insertion operator (operator<<).

3.7 Attendance (abstract capture with group choice)

Attendance core (mandatory, identical for all groups). Every group implements the same attendance domain model:

- FR7.1 – A Lecturer shall be able to open an attendance session for one of their own course time slots and close it manually or by expiry (configurable duration, e.g., 10 minutes).

- FR7.2 – A student may be marked present only if the session is open, the student is enrolled in that course, and the student has not already been marked in that session. Each violation shall raise a distinct custom exception (e.g., SessionClosedException, NotEnrolledException, DuplicateAttendanceException).
- FR7.3 – Attendance records shall be immutable: a record stores the student, session, timestamp, status (present/late), and how it was captured. Corrections are made by appending a correction record (with the acting lecturer and reason), never by editing or deleting.
- FR7.4 – The system shall report per-student and per-course attendance percentages. (Optional extension: an eligibility report listing students below a configurable threshold, default 80%.)
- FR7.5 – Attendance data shall persist and reload with all other system data.

Capture mechanism (group choice via an abstract interface). How a 'present' event enters the system must be hidden behind an abstract class:

- FR7.6 – The group shall define an abstract AttendanceCapture class with pure virtual functions (e.g., beginSession(), captureNext(), endSession()). The attendance core shall depend only on this interface — it must not know which concrete mechanism is in use.
- FR7.7 – The group shall implement at least TWO concrete capture classes: (a) FileReplayCapture — mandatory for all groups — which replays capture events from a text file (used for testing and bulk demonstration, including malformed lines that trigger exceptions); and (b) ONE interactive mechanism chosen by the group from the approved list below.
- FR7.8 – The active capture mechanism shall be selectable at runtime (e.g., a lecturer setting), demonstrating that concrete captures are interchangeable through base-class pointers/references.

Approved interactive capture mechanisms (choose one):

  - Option A – Rotating session code: the system displays a short random code with an expiry; students enter the code from their own menu to check in.
  - Option B – Simulated card tap: each student owns a StudentCard with a unique UID; a ConsoleCardReader reads UIDs typed (or scanned by a USB reader acting as a keyboard) at the lecturer's station.
  - Option C – QR token: the system generates a session token (session ID + expiry + integrity hash) rendered as a QR code in the terminal using a single-file QR library (e.g., Nayuki's qrcodegen, MIT licence); students submit the decoded payload string, which the system validates.
  - Option D – Roll-call with audit trail: the lecturer marks each student interactively; the emphasis shifts to the correction/audit-trail records of FR7.3, which must then support a full change-history report.

Your chosen option must be named and justified in the D1 design rationale, and your UML must show the AttendanceCapture hierarchy. Optional bonus (up to +5 marks, capped at 100): a second interactive capture mechanism, or real hardware/webcam capture (e.g., OpenCV/ZBar QR scanning), provided the interface design is unchanged.

4. Mandatory OOP and Technical Requirements

Regardless of extensions, your implementation must demonstrably include all of the following. Each item will be inspected in the code review and probed at the viva.

Concept	Minimum requirement
Encapsulation	No public data members in domain classes; all state accessed through a deliberate public interface. Business rules enforced inside classes, not in the menu code.
Inheritance & polymorphism	At least two inheritance hierarchies (e.g., Person and Course) with an abstract base class, virtual functions overridden in derived classes, and at least one place

	where objects are used polymorphically through base-class pointers or references. Base classes must declare virtual destructors.
Abstract interfaces & dependency inversion	The AttendanceCapture abstract class (pure virtual functions, virtual destructor) with at least two concrete implementations, selected and used at runtime through base-class pointers/references. The attendance core must compile without knowledge of any concrete capture class.
Composition / aggregation	At least one composition relationship (e.g., Course owns its AttendanceRegister and Timetable owns TimeSlot objects) and one aggregation relationship (e.g., Course refers to enrolled Students), and you must be able to explain the difference in your design.
Constructors / destructors / Rule of Three or Five	At least one class manages a dynamic resource and correctly implements the destructor, copy constructor, and copy assignment operator (or explicitly deletes copying with justification).
Operator overloading	At least three meaningful overloaded operators, including operator<< for output and one comparison/logic operator used in clash detection.
Exception handling	A small hierarchy of custom exception classes derived from std::exception, thrown by the domain layer and caught at the user-interface layer.
Templates & STL	Appropriate use of STL containers (vector, map, etc.) and at least one group-written class or function template (e.g., a generic Repository<T>).
File handling	Object state saved and restored through fstream, encapsulated behind a storage interface.
Static members / const-correctness	At least one justified use of a static member; getter functions and read-only parameters declared const.
Code organisation	Each class split into a header (.h) and implementation (.cpp) file with include guards; the project builds with a provided Makefile or CMakeLists.txt using g++ -std=c++17 -Wall.

5. Group Roles and Responsibilities

Each group has exactly three members. The suggested division below gives every member both design and implementation responsibility. You may adjust boundaries, but the final report must contain a contribution table stating who designed, implemented, and tested each class, and every member must be able to explain the whole design at the viva.

Member	Responsibility
Member 1 – Domain model	Person hierarchy (Student, Lecturer, Administrator), Course hierarchy, enrolment rules, custom exception hierarchy.
Member 2 – Scheduling & attendance engine	TimeSlot and Timetable classes, operator-based clash detection, prerequisite checking, the attendance core (AttendanceSession, AttendanceRecord, AttendanceRegister, validation exceptions), the abstract AttendanceCapture interface with the group's chosen concrete capture, and attendance reporting.
Member 3 – Persistence & UI	Storage/Repository classes and file formats (including attendance persistence and the FileReplayCapture), template Repository<T> if used, menu system, integration, build scripts, and system testing.

Important: interfaces between the three parts must be agreed and frozen at the design checkpoint. This is deliberate — it forces you to design before you code.

6. Deliverables and Timeline (4 Weeks)

When	Deliverable	Contents
Week 1, Day 2	Group registration	Group members, initial role allocation, and provisional attendance capture option. Submitted via the LMS form.
End of Week 2	D1 – Design checkpoint (assessed)	UML class diagram (including the AttendanceCapture hierarchy), short design rationale (max 3 pages), and interface agreement between members. Presented in a 10-minute design review meeting.
Week 4, Day 3	D2 – Final submission (assessed)	Full source code, build script, sample data files, final UML diagram, design change log, and project report. Submitted as a single zip via the LMS.
End of Week 4	D3 – Viva & demonstration (assessed)	20-minute group demonstration followed by individual viva questions to each member.

Pace warning: this is a four-week project. The design checkpoint falls at the end of Week 1, which leaves no slack for late group formation — agree your roles and capture option in the first two days and start the UML immediately.

7. The UML Design Checkpoint (D1)

7.1 What you must submit

- A complete UML class diagram of your intended system, produced with a proper tool (draw.io, PlantUML, StarUML, Visual Paradigm, or similar — no hand-drawn sketches). Export as PDF or PNG at readable resolution.
- The diagram must show: all classes with key attributes and operations; visibility markers (+, −, #); inheritance (generalisation) arrows; composition and aggregation with correct diamond notation; associations with multiplicities; abstract classes/pure virtual functions in italics or marked «abstract»; and template classes if planned.
- A design rationale of at most three pages explaining: why each hierarchy exists, where polymorphism will be used, which relationships are composition vs aggregation and why, where each mandatory OOP requirement from Section 4 will appear, which interactive attendance capture option (Section 3.7) you chose and why, and how the design maps to the three member roles.
- The AttendanceCapture hierarchy must appear in the diagram: the abstract class with its pure virtual operations, the FileReplayCapture, your chosen interactive capture, and the dependency showing that the attendance core uses only the abstract interface.
- An interface agreement: a one-page table of the public functions each member's part exposes to the others.

7.2 What happens at the checkpoint

Each group attends a 10-minute design review. You will walk through the diagram, and the examiner will probe design decisions (e.g., "Why is Timetable a composition and not an aggregation?", "Which function is polymorphic here?"). You will receive feedback and have to submit final diagram addressing the comments.

The approved diagram is frozen as your design baseline. A copy is retained by the module staff and used for the conformance check at final marking.

7.3 Design changes after the checkpoint

Designs legitimately evolve during implementation. Changes are allowed, but every deviation from the approved baseline must be recorded in a design change log submitted with the final code. Each entry must state: what changed (class/relationship/function), why, and which member approved it. Sensible, well-justified evolution is rewarded; silent divergence between diagram and code is penalised (see Section 8).

8. Design Conformance Check (at Final Marking)

At final marking, your submitted code will be compared against your approved Week-1 UML baseline plus your change log, using the following procedure:

1. Class inventory: every class in the baseline exists in code (or its removal is in the change log), and every significant class in code appears in the final UML.
2. Relationship check: inheritance, composition, and aggregation relationships in the code match the diagram — e.g., if the diagram shows Timetable composed of TimeSlot, the code must reflect ownership (not a shared pointer to slots owned elsewhere).
3. Interface check: public operations of key classes match the diagram's operations (renames and small signature changes are fine if logged).
4. Change log audit: each substantive difference is matched to a change-log entry with a plausible justification.
5. Final UML accuracy: the updated diagram submitted with the final code truthfully reflects the implemented system.

Conformance carries its own weight in the rubric (Section 11). A group that changed its design thoughtfully and documented it can score full conformance marks; a group whose code matches the diagram only by accident, or whose diagram was clearly reverse-engineered at the last minute without a change history, cannot.

9. Final Submission Requirements (D2)

9.1 Code and build

- Source code organised into include/ and src/ folders (or per-member modules), one class per header/implementation pair.
- A Makefile or CMakeLists.txt; the project must build on a clean machine with g++ -std=c++17 -Wall with no errors. Code that does not compile is capped at 40% of the implementation component.
- Sample data files sufficient to demonstrate every feature, plus a README with build and run instructions and test login credentials.
- Consistent naming and formatting; meaningful comments on class responsibilities and non-obvious logic (not line-by-line narration).

9.2 Project report (6–8 pages)

- Final UML class diagram (updated to match the code).
- Design change log (Section 7.3).
- A concept-mapping table: for each mandatory requirement in Section 4, the file, class, and line references where it is implemented.
- Brief description of the file format used for persistence.
- Testing summary: the scenarios you tested, including at least three exception paths, one clash-detection case, and a FileReplayCapture run containing both valid and malformed attendance events.
- Individual contribution table signed by all members.

- Known limitations and possible improvements.

10. Academic Integrity and Use of AI Tools

- This is a group assessment: collaboration inside your group is expected; sharing designs or code between groups is plagiarism. Submissions are compared with each other, with online sources, and with previous years' submissions.
- AI assistants (ChatGPT, Claude, Copilot, etc.) may be used the way you would use a textbook or a lab demonstrator: to explain concepts, suggest approaches, and help debug. Submitting substantially AI-generated code as your own work is not permitted.
- You must declare AI usage in the report (which tools, for what). The viva is the control: every member will be asked to explain and modify parts of the code live. Inability to explain your own submission will be treated as evidence of academic misconduct and marked accordingly.
- Late submission penalty: 10 percentage points per working day, up to three days, after which the submission receives zero.

11. Marking Rubric

The project is marked out of 100 as follows. The design checkpoint (D1) is marked at the end of Week 1 and its mark is fixed at that point (except where a resubmission penalty applies). Detailed band descriptors follow on the next pages.

Component	Marked at	Weight
A. Design checkpoint – UML diagram, rationale & design review (group)	Week 2	20%
B. Implementation – functionality & OOP quality (group)	Week 4	35%
C. Design conformance – code vs approved UML + change log (group)	Week 4	10%
D. Code quality & build (group)	Week 4	10%
E. Project report (group)	Week 4	10%
F. Viva & demonstration (individual)	Week 4	15%
Total		100%

Individual adjustment: group components (A–E) may be scaled per member by up to ±20% based on the contribution table, commit history (if a shared repository is used), peer assessment forms, and viva performance, where evidence shows clearly unequal contribution.

Appendix A: Detailed Band Descriptors

A. Design Checkpoint – 20 marks

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
A1. UML notation & completeness	6	All classes, attributes, operations, visibility, multiplicities, and relationship notation correct; abstract classes and key virtual operations clearly marked; diagram is readable and tool-produced.	Largely complete and correct; minor notation errors (e.g., a missing multiplicity or wrong diamond) that do not obscure the design.	Main classes and inheritance shown but relationships incomplete or notation frequently incorrect; some attributes/operations missing.	Sketchy or hand-waved diagram; notation largely wrong; cannot serve as an implementation baseline.
A2. Quality of the OO design	8	Clear separation of concerns; sensible hierarchies with genuine polymorphic behaviour; correct composition vs aggregation choices; every Section-4 requirement has an identified home; no god-classes.	Sound overall structure with one or two questionable decisions (e.g., a thin subclass, one overloaded responsibility); mandatory concepts mostly placed.	Design works but is procedural in places (logic pushed into menu/manager classes); hierarchy exists but adds little; several mandatory concepts unplaced.	Little evidence of design thinking; classes are data buckets; hierarchy artificial or absent.
A3. Design rationale & interface agreement	3	Rationale convincingly justifies each major decision; interface agreement is precise (names, parameters, ownership) and covers all cross-member boundaries.	Reasonable justification of most decisions; interfaces defined but with some gaps.	Rationale descriptive rather than justificatory; interfaces vague.	Rationale missing or restates the diagram; no usable interface agreement.
A4. Design review performance	3	All members can explain the design and answer probing questions; decisions defended with correct terminology.	Most questions answered correctly; minor hesitation or one member noticeably weaker.	Only one member can defend the design; answers superficial.	Group cannot explain its own diagram.

B. Implementation – 35 marks

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
B1. Functional completeness & correctness	12	All functional requirements (FR1–FR7) work correctly, including prerequisite chains, clash detection, attendance sessions with both capture mechanisms, attendance reporting, and persistence of all relationships; edge cases handled.	All core flows work; one or two secondary requirements weak or buggy (e.g., a report missing, an edge case unhandled).	Basic flows (login, enrol, view timetable) work but several requirements incomplete or unreliable; persistence partial.	Major features missing or frequently failing; system unusable for its core purpose.
B2. Inheritance, polymorphism & class design in code	8	Hierarchies match good design: abstract bases, virtual destructors, overridden behaviour genuinely used through base-class pointers/references; capture mechanisms swap at runtime purely through the AttendanceCapture interface; no role-string if/else chains.	Hierarchies present and mostly used polymorphically; occasional type-switching or a redundant subclass.	Inheritance exists but behaviour selected mainly by conditionals; polymorphism token or unused.	No meaningful inheritance/polymorphism; monolithic or procedural code.
B3. Encapsulation, composition & resource management	7	No public data; invariants enforced inside classes; composition/aggregation implemented with correct ownership; Rule of Three/Five class correct (or copying correctly deleted with justification); no leaks in normal flows.	Encapsulation generally respected with minor leaks of logic to the UI layer; resource management correct with small omissions.	Frequent getters/setters exposing everything; ownership unclear; copy semantics untested or partly wrong.	Public data members; business rules in main(); broken or absent resource management.
B4. Exceptions, operators, templates & STL	8	Custom exception hierarchy used across domain/UI boundary; all three operators meaningful and idiomatic; group-written template genuinely reused; STL containers chosen appropriately.	All mechanisms present and mostly idiomatic; one used superficially (e.g., a template instantiated once for show).	Mechanisms present but token: exceptions thrown and caught in the same function, operators trivial, STL used where a raw array mindset persists.	One or more mandatory mechanisms absent.

C. Design Conformance – 10 marks

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
C1. Code matches approved baseline + change log	6	Every class, relationship, and key interface either matches the approved UML or is covered by a justified change-log entry; ownership semantics (composition vs aggregation) implemented as drawn.	Code substantially matches; a few small undocumented drifts (renames, added helpers) that don't alter the architecture.	Recognisably related to the baseline but with several undocumented structural changes.	Code bears little relation to the approved design; no credible change history.
C2. Change log & final UML accuracy	4	Change log complete, dated, and justified; final UML precisely reflects the implemented system.	Log covers major changes; final UML accurate with minor omissions.	Log sparse or vague; final UML partially out of date.	No change log; final UML missing or clearly reverse-engineered and still wrong.

D. Code Quality & Build – 10 marks

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
D1. Build & organisation	5	Builds cleanly with g++ -std=c++17 -Wall (no warnings); proper header/implementation split with include guards; working Makefile/CMake; clear README and sample data.	Builds with minor warnings; organisation good with small inconsistencies.	Builds only after manual steps; organisation messy (logic in headers, missing guards).	Does not compile as submitted (implementation component capped at 40%).
D2. Readability & style	5	Consistent naming and formatting; const-correct; comments explain intent; no dead code or debug litter.	Generally readable; some inconsistency or leftover debug output.	Hard to follow in places; sparse or misleading comments.	Unreadable, chaotic, or clearly uncleaned generated code.

E. Project Report – 10 marks

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
E1. Concept mapping & testing evidence	5	Concept-mapping table complete and accurate to file/class level; testing summary covers all required scenarios with observed results.	Mapping mostly complete; testing covers main paths.	Mapping partial or inaccurate; testing anecdotal.	Mapping/testing missing.
E2. Report quality & completeness	5	All required sections present, well written, honest about limitations; contribution table specific and signed.	Complete with minor gaps or padding.	Several sections thin or missing; contribution table vague.	Report perfunctory or absent.

F. Viva & Demonstration – 15 marks (individual)

Criterion	Wt.	Excellent (70–100%)	Good (55–69%)	Satisfactory (40–54%)	Poor (0–39%)
F1. Demonstration	5	Confident, complete demo covering all features including failure/exception paths, within time.	Smooth demo of core features; minor fumbling or a missed feature.	Demo works only for happy paths; poorly rehearsed.	Demo fails or cannot show core features.
F2. Individual understanding	7	Explains any part of the design and code, including parts owned by teammates; answers 'what if we changed X' questions correctly; correct OOP terminology throughout.	Explains own part fully and teammates' parts at a high level; most probing questions answered.	Explains own code descriptively but struggles with 'why' questions or with the rest of the system.	Cannot explain code attributed to them (triggers academic-integrity review).
F3. Live modification task	3	Correctly performs a small live change (e.g., add a rule, override a function) explaining each step.	Completes the change with minor help.	Attempts the change with significant guidance; partially correct.	Unable to attempt the change.

Appendix B: Peer Assessment

Each member privately submits a peer assessment form with the final submission, rating each teammate (including themselves) on contribution, reliability, and quality of work, with brief comments. Peer assessments, the contribution table, and repository history inform the ±20% individual scaling described in Section 11. Peer forms are confidential to module staff.